Universidad Rey Juan Carlos

Aplicaciones Industriales de Visión Artificial

Conteo y segmentación de glóbulos rojos en imágenes de plasma sanguíneo

Documento de diseño y funcionalidades principales

Proyecto: AIVA_2026-RBC_Counter

Autores: Daniel Cámara Núñez y Carla Barbero Martín

Fecha: 12 de Abril 2026

Índice

1. Introducción **2**
1.1. Tests con ejemplos 2
1.2. Descripción de técnicas de visión 2
1.3. Referencias 3

2. Diseño de la solución **3**
2.1. Diagramas de clases 3
2.1.1. Diagrama de clases de entrenamiento 3
2.1.2. Diagrama de clases del sistema 5
2.2. Diagramas de secuencia 9
2.2.1. Diagrama de secuencia de la aplicación 9
2.2.2. Diagrama de secuencia del sistema de visión 10
2.3. Diagramas de actividad de los algoritmos 11
2.3.1. Funcionamiento general del sistema 11
2.3.2. Algoritmo de visión (YOLO) 12
2.3.3. Entrenamiento del modelo 13
2.3.4. Evaluación del modelo 14

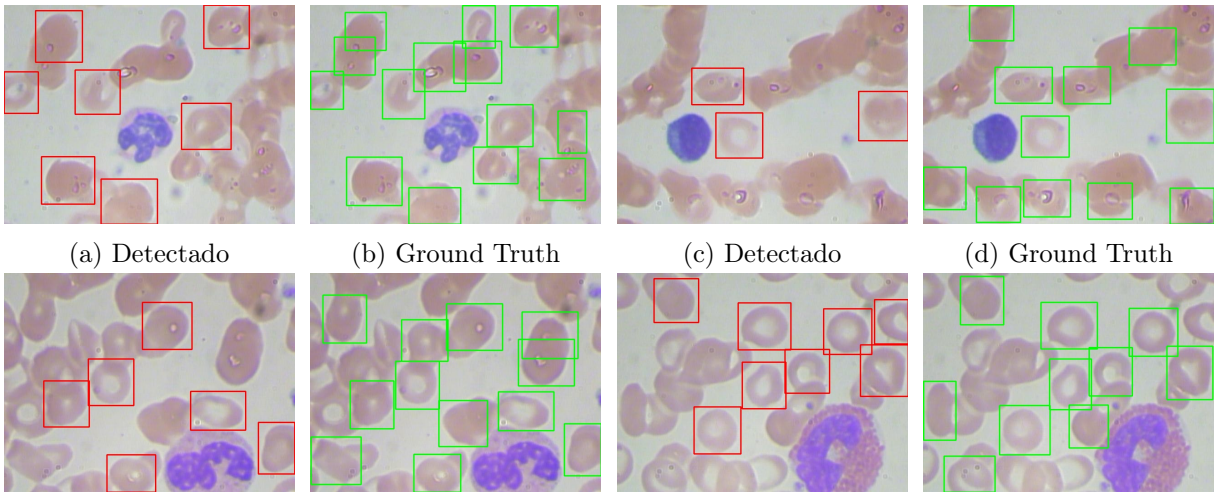
1. Introducción

El presente documento describe el desarrollo de un sistema de visión artificial orientado al conteo y segmentación automática de glóbulos rojos en imágenes de plasma sanguíneo, cuyo objetivo es automatizar el análisis de muestras, sustituyendo el conteo manual realizado por personal especializado. Para ello, se ha diseñado un pipeline de procesamiento de imágenes que permite detectar las células presentes en la muestra, segmentarlas mediante bounding boxes y calcular su número total a partir de imágenes obtenidas mediante microscopio, aplicando diferentes técnicas de visión artificial. Como resultado, el sistema genera un archivo estructurado con la información de las células detectadas, junto con las imágenes procesadas, facilitando su almacenamiento y posterior análisis, e integrando tanto el diseño de la solución como los principales algoritmos empleados.

Repositorio donde se desarrollará el sistema:
`https://github.com/DaniCamaraN/AIVA_2026-RBC_Counter`

1.1. Tests con ejemplos

Para validar el funcionamiento del sistema desarrollado, se han realizado pruebas sobre distintas imágenes de muestras de plasma sanguíneo obtenidas mediante microscopio. Estas pruebas permiten evaluar la capacidad del sistema para detectar, segmentar y contar correctamente los glóbulos rojos presentes en cada imagen. A continuación, se muestran algunos ejemplos representativos junto con los resultados obtenidos.



(a) Detectado (b) Ground Truth (c) Detectado (d) Ground Truth

(e) Detectado (f) Ground Truth (g) Detectado (h) Ground Truth

Figura 1: Ejemplo de comparación entre detecciones y ground truth.

1.2. Descripción de técnicas de visión

El sistema desarrollado emplea técnicas de visión artificial basadas en detección de objetos mediante aprendizaje profundo, complementadas con etapas de procesamiento de imagen para preparar los datos de entrada.

En primer lugar, se realiza una etapa de preprocesado de la imagen, cuyo objetivo es mejorar la calidad visual y facilitar la detección posterior. Dependiendo de la implementación, esta fase puede incluir operaciones como conversión de espacio de color, normalización de intensidades o suavizado mediante filtros gaussianos para reducir el ruido.

La técnica principal utilizada es la detección de objetos mediante redes neuronales convolucionales, concretamente mediante un modelo basado en la arquitectura YOLO (You Only Look

Once). Este tipo de modelos divide la imagen en una cuadrícula y predice directamente, para cada región, la presencia de objetos junto con sus bounding boxes y probabilidades de clase. De este modo, se obtiene la localización de cada glóbulo rojo en la imagen en una única pasada por la red. Las detecciones generadas por el modelo se representan mediante bounding boxes, definidas por sus coordenadas en la imagen. A partir de estas bounding boxes, el sistema realiza el conteo de glóbulos rojos, considerando cada detección válida como una célula individual.

Finalmente, los resultados se estructuran en un formato de salida que incluye tanto la información geométrica de las detecciones como su representación visual, permitiendo la validación y análisis posterior.

1.3. Referencias

ID	Referencia
REF-01	Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Cuadro 1: Referencias de las técnicas de visión

2. Diseño de la solución

2.1. Diagramas de clases

2.1.1. Diagrama de clases de entrenamiento

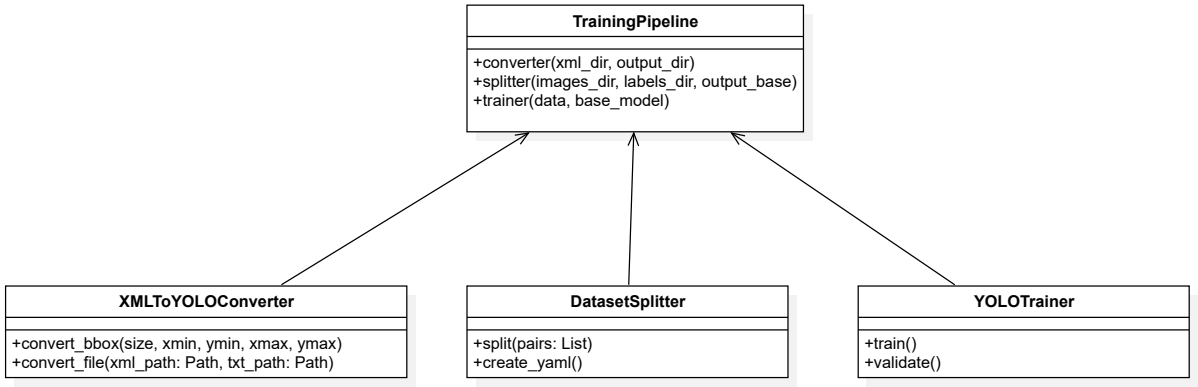


Figura 2: Diagrama UML de clases del entrenamiento

El diagrama de la figura 2 representa la arquitectura del sistema encargado del entrenamiento del modelo de detección de glóbulos rojos. En él se define una estructura modular en la que cada componente tiene una responsabilidad específica dentro del flujo de preparación de datos y entrenamiento.

La clase principal es `TrainingPipeline`, que actúa como elemento orquestador del proceso completo. El pipeline centraliza la ejecución secuencial de las distintas etapas necesarias para entrenar el modelo.

En primer lugar, la clase `XMLToYOLOConverter` se encarga de transformar las anotaciones en formato XML al formato requerido por YOLO, adaptando las coordenadas de las bounding boxes y la estructura de los ficheros. A continuación, la clase `DatasetSplitter` gestiona la preparación del dataset, dividiéndolo en conjuntos de entrenamiento y test, y generando el archivo de configuración necesario para el modelo. Por último, la clase `YOLOTrainer` es responsable del entrenamiento y validación del modelo, encapsulando la interacción con la librería YOLO.

DatasetSplitter

La clase `DatasetSplitter` maneja la división del dataset en conjuntos de entrenamiento y validación para el entrenamiento de modelos de machine learning. Utiliza la biblioteca *pathlib* para manejar rutas de archivos y *shutil* para copiar archivos, asegurando una distribución aleatoria pero reproducible de los datos. Esta clase es crucial para preparar datasets balanceados que eviten el sobreajuste durante el entrenamiento.

Los métodos principales incluyen:

- `__init__(images_dir: str, labels_dir: str, output_base: str, train_ratio: float, seed: int)`: para configurar las rutas y parámetros.
- `collect_pairs()`: para emparejar imágenes con sus etiquetas.
- `split(pairs)`: para dividir en train/val.
- `create_yaml()`: para generar el archivo de configuración YOLO.
- `run()`: para ejecutar todo el proceso de división.

XMLToYOLOConverter

La clase `XMLToYOLOConverter` convierte anotaciones en formato *XML (PASCAL VOC)* al formato de texto plano requerido por YOLO. Realiza la transformación de coordenadas absolutas a normalizadas relativas al tamaño de la imagen, y mapea nombres de clases a identificadores numéricos. Esta conversión es necesaria porque YOLO utiliza un formato de anotación diferente al de los datasets de ground truth originales.

Los métodos principales son:

- `__init__(xml_dir: str, output_dir: str, class_map: Dict[str, int])`: para inicializar con rutas y mapeo de clases.
- `convert_bbox(size, xmin, ymin, xmax, ymax)`: para transformar coordenadas.
- `convert_file(xml_path: Path, txt_path: Path)`: para procesar un archivo individual.
- `run()`: para convertir todos los archivos XML en el directorio.

YOLOTrainer

La clase `YOLOTrainer` encapsula la funcionalidad de entrenamiento de modelos YOLO utilizando la biblioteca *Ultralytics*. Configura todos los parámetros necesarios para el entrenamiento, incluyendo el modelo base, número de épocas, tamaño de imagen y configuración de dispositivo (CPU/GPU). Proporciona una interfaz simplificada para entrenar, validar y cargar modelos entrenados.

Los métodos principales incluyen:

- `__init__(data_yaml: str, base_model: str, epochs: int, imgsz: int, batch: int, project: str, name: str, device: Optional[str])`: configura el entrenador.
- `train()`: ejecuta el entrenamiento.
- `validate()`: valida el modelo.
- `get_best_model_path()`: obtiene la ruta del mejor modelo guardado.
- `load_best_model()`: carga el modelo entrenado.

TrainingPipeline

La clase `TrainingPipeline` orquesta todo el proceso de preparación y entrenamiento de modelos YOLO para la detección de glóbulos rojos. Combina la conversión de datos, división del dataset y entrenamiento en una secuencia automatizada, facilitando el flujo completo desde datos crudos hasta un modelo entrenado listo para usar.

Los métodos principales son:

- `__init__()`: inicializa los componentes (conversor, divisor y entrenador).
- `run()`: ejecuta la secuencia completa: conversión XML a YOLO, división del dataset y entrenamiento del modelo.

2.1.2. Diagrama de clases del sistema

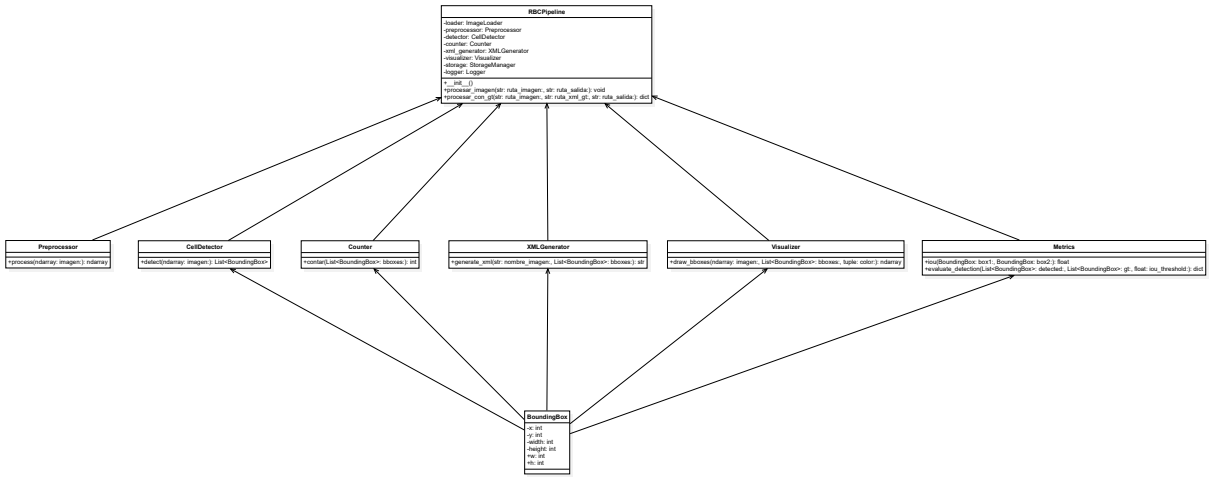


Figura 3: Diagrama UML de clases del sistema

El diagrama de clases de la figura 3 representa la estructura estática del sistema desarrollado para la detección y conteo de glóbulos rojos. La clase principal es `RBCPipeline`, que actúa como núcleo del sistema y coordina el funcionamiento de los distintos módulos encargados de cada etapa del procesamiento. En ella se agrupan como atributos las clases `Preprocessor`, `CellDetector`, `Counter`, `XMLGenerator` y `Visualizer`, reflejando una arquitectura modular.

Las imágenes son procesadas inicialmente por `Preprocessor`, que prepara la imagen para su análisis. A continuación, `CellDetector` aplica el algoritmo de visión artificial para obtener las detecciones en forma de bounding boxes. Posteriormente, `Counter` calcula el número de células detectadas, mientras que `XMLGenerator` genera el archivo de salida y `Visualizer` permite representar visualmente las detecciones.

Por último, la clase `Metrics` implementa las funciones necesarias para evaluar la detección mediante IoU y comparar los resultados con el ground truth, siendo utilizada en el método `procesar_con_gt` de `RBCPipeline`.

Bounding Box

La clase `BoundingBox` es una estructura de datos simple implementada como una dataclass en *Python*, diseñada para representar las coordenadas y dimensiones de una caja delimitadora rectangular en una imagen. Esta clase encapsula la información espacial necesaria para identificar y localizar objetos detectados, en este caso glóbulos rojos, proporcionando una interfaz consistente para el manejo de posiciones y tamaños a lo largo del sistema.

Los métodos principales incluyen propiedades para acceder y modificar las coordenadas x e y, así como *width* y *height*, con aliases w y h para mayor comodidad. No tiene métodos adicionales complejos, ya que su propósito es puramente de almacenamiento de datos.

AnnotationReader

La clase `AnnotationReader` se encarga de leer archivos XML que contienen anotaciones de *ground truth* para las imágenes del dataset. Utiliza el módulo *xml.etree.ElementTree* de *Python* para parsear los archivos XML y extraer las coordenadas de las cajas delimitadoras anotadas manualmente, convirtiéndolas en instancias de *BoundingBox*. Esta clase es fundamental para la evaluación del rendimiento del sistema de detección, ya que proporciona los datos de referencia.

El método principal es `from_xml(path: str)`, que toma la ruta a un archivo XML y devuelve una lista de objetos BoundingBox representando las anotaciones encontradas en el archivo.

ImageLoader

La clase `ImageLoader` maneja la carga de imágenes desde el sistema de archivos utilizando la biblioteca *OpenCV*. Su función principal es abstraer el proceso de lectura de imágenes, asegurando que se carguen correctamente y manejando errores en caso de que el archivo no exista o no se pueda leer.

El método principal es `load_image(image_filename: str)`, que recibe la ruta del archivo de imagen y devuelve un array *NumPy* que representa la imagen en formato BGR, listo para ser procesado por otros componentes del sistema.

Preprocessor

La clase `Preprocessor` aplica transformaciones básicas de preprocesamiento a las imágenes para mejorar la calidad de la detección posterior. Convierte la imagen a escala de grises para simplificar el análisis y aplica un filtro de suavizado Gaussiano para reducir el ruido, lo que facilita la detección de bordes y contornos en etapas posteriores del pipeline.

El método principal es `process(imagen: ndarray)`, que toma una imagen en formato BGR y devuelve la imagen preprocesada en escala de grises y suavizada.

CellDetector

La clase `CellDetector` implementa la detección de glóbulos rojos utilizando el framework *YOLO (You Only Look Once)* de *Ultralytics*, un modelo de deep learning preentrenado para detección de objetos. Este enfoque, basado en redes neuronales convolucionales, proporciona una gran precisión y robustez en la detección automática de células.

Los métodos principales incluyen:

- `__init__(model_path: str, conf_threshold: float, class_id: Optional[int])`: para inicializar el modelo YOLO con parámetros configurables como la ruta del modelo, el umbral de confianza y el ID de clase.
- `detect(imagen: ndarray)`: que procesa una imagen y devuelve una lista de objetos *BoundingBox* correspondientes a las células detectadas.

Counter

La clase `Counter` proporciona funcionalidad básica para contar el número de objetos detectados. Su implementación es simple, cuantifica los resultados del proceso de detección, permitiendo evaluar la efectividad del algoritmo en términos de cantidad de células identificadas.

El método principal es `contar(bboxes: List[BoundingBox])`, que recibe una lista de cajas delimitadoras y devuelve el número total de elementos en la lista.

XMLGenerator

La clase `XMLGenerator` se encarga de generar archivos XML estructurados que contienen las detecciones realizadas por el sistema. Crea un formato XML personalizado que incluye el nombre de la imagen y las coordenadas de todas las cajas delimitadoras detectadas, facilitando el almacenamiento y posterior análisis de los resultados.

El método principal es `generate_xml(nombre_imagen: str, bboxes: List[BoundingBox])`, que toma el nombre de la imagen y una lista de cajas delimitadoras, y devuelve una cadena de texto con el contenido XML generado.

Visualizer

La clase `Visualizer` maneja la visualización de resultados superponiendo cajas delimitadoras sobre las imágenes originales. Utiliza *OpenCV* para dibujar rectángulos coloreados que representan las detecciones, permitiendo una evaluación visual rápida del rendimiento del algoritmo y facilitando la comparación entre detecciones automáticas y anotaciones manuales.

El método principal es `draw_bboxes(imagen: ndarray, bboxes: List[BoundingBox], color: tuple)`, que recibe una imagen y una lista de cajas delimitadoras, y devuelve la imagen con las cajas dibujadas en el color especificado (por defecto verde).

StorageManager

La clase `StorageManager` coordina el guardado de todos los resultados del procesamiento, incluyendo la imagen original, el XML generado y múltiples versiones de overlays visuales. Crea la estructura de directorios necesaria y maneja el almacenamiento eficiente de archivos, permitiendo guardar diferentes visualizaciones (como detecciones y ground truth) con sufijos descriptivos.

El método principal es `save_results(imagen: ndarray, xml: str, overlays: List[Tuple], ruta_salida: str, nombre_base: str)`, que guarda la imagen, XML y overlays en la ruta especificada con el nombre base proporcionado.

Logger

La clase `Logger` proporciona funcionalidad de *logging* para rastrear el progreso y errores durante la ejecución del sistema. Utiliza el módulo *logging* de *Python* para escribir mensajes a un archivo de log, permitiendo monitorear el comportamiento del pipeline y diagnosticar problemas durante el procesamiento.

Los métodos principales son:

- `__init__(ruta_log: str)`: para inicializar el logger con la ruta del archivo de log.
- `info(mensaje: str)`: para registrar mensajes informativos, y `error(mensaje: str)` para registrar errores.

Metrics

La clase `Metrics` contiene funciones utilitarias para evaluar el rendimiento del sistema de detección comparando las cajas delimitadoras detectadas con las anotaciones de ground truth. Calcula métricas estándar como precisión, recall, y cuenta de verdaderos positivos, falsos positivos y falsos negativos, utilizando el cálculo de *Intersection over Union (IoU)* para determinar coincidencias.

Los métodos principales son:

- `iou(box1: BoundingBox, box2: BoundingBox)`: para calcular la intersección sobre unión entre dos cajas.
- `evaluate_detection(detected: List[BoundingBox], gt: List[BoundingBox], iou_threshold: float)`: para evaluar una detección completa y devolver un diccionario con todas las métricas.

RBCPipeline

La clase `RBCPipeline` actúa como el orquestador principal del sistema, coordinando todas las demás clases para ejecutar el proceso completo de detección de glóbulos rojos. Implementa el patrón de diseño *Pipeline*, donde cada etapa del procesamiento (carga, preprocesamiento, detección, conteo, etc.) se delega a clases especializadas. Esto proporciona una arquitectura modular y extensible.

Los métodos principales son:

- `__init__()`: para inicializar todas las dependencias.
- `procesar_imagen(ruta_imagen: str, ruta_salida: str)`: para procesar una imagen sin ground truth.
- `procesar_con_gt(ruta_imagen: str, ruta_xml_gt: str, ruta_salida: str)`: para procesar con comparación contra anotaciones manuales y devolver métricas de evaluación.

2.2. Diagramas de secuencia

2.2.1. Diagrama de secuencia de la aplicación

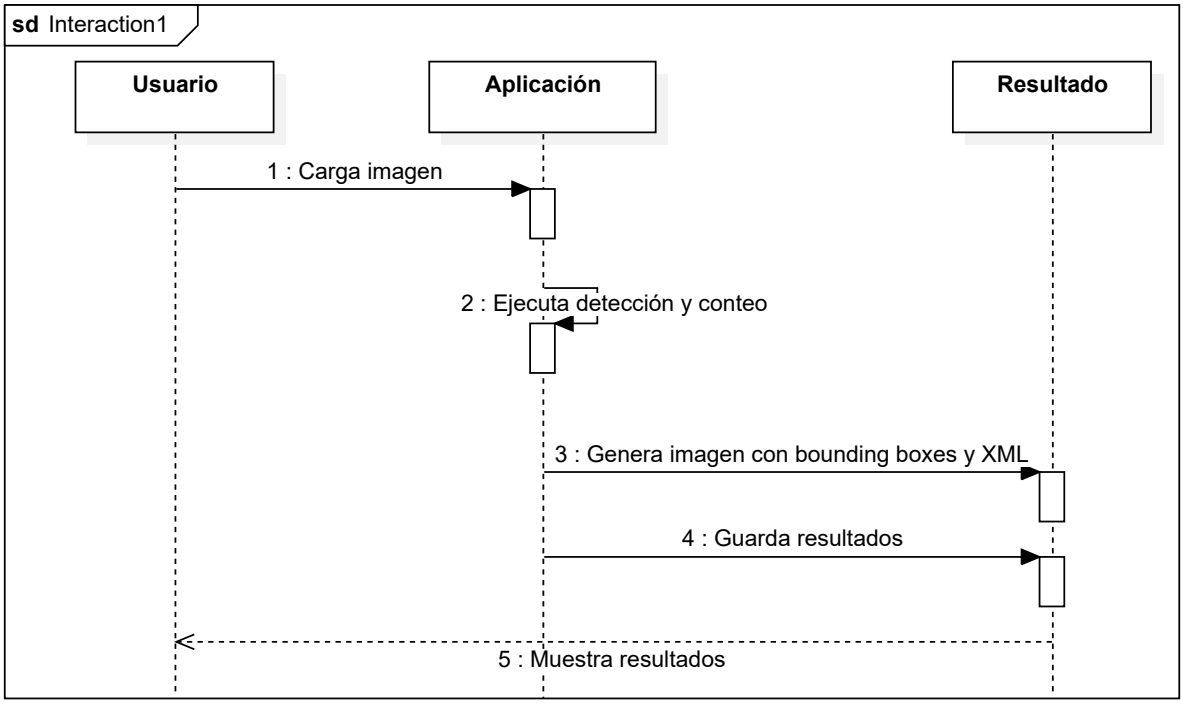


Figura 4: Diagrama UML de secuencia de la aplicación

El diagrama de secuencia de la figura 4 representa el flujo de uso de la aplicación desde el punto de vista del usuario, mostrando la interacción entre el usuario y el sistema.

El proceso comienza cuando el usuario carga una imagen en la aplicación o cuando esta se recibe desde el microscopio, en el caso de trabajar en tiempo real. A continuación, el sistema procesa la imagen ejecutando el algoritmo de detección y conteo de glóbulos rojos. Posteriormente, se generan las salidas correspondientes, que incluyen la imagen con las bounding boxes y la información en formato XML. Estas salidas se almacenan y finalmente se presentan al usuario para su análisis.

Este diagrama permite entender de forma clara el funcionamiento general del sistema desde el punto de vista del usuario, destacando las principales entradas y salidas sin mostrar los detalles internos de implementación.

2.2.2. Diagrama de secuencia del sistema de visión

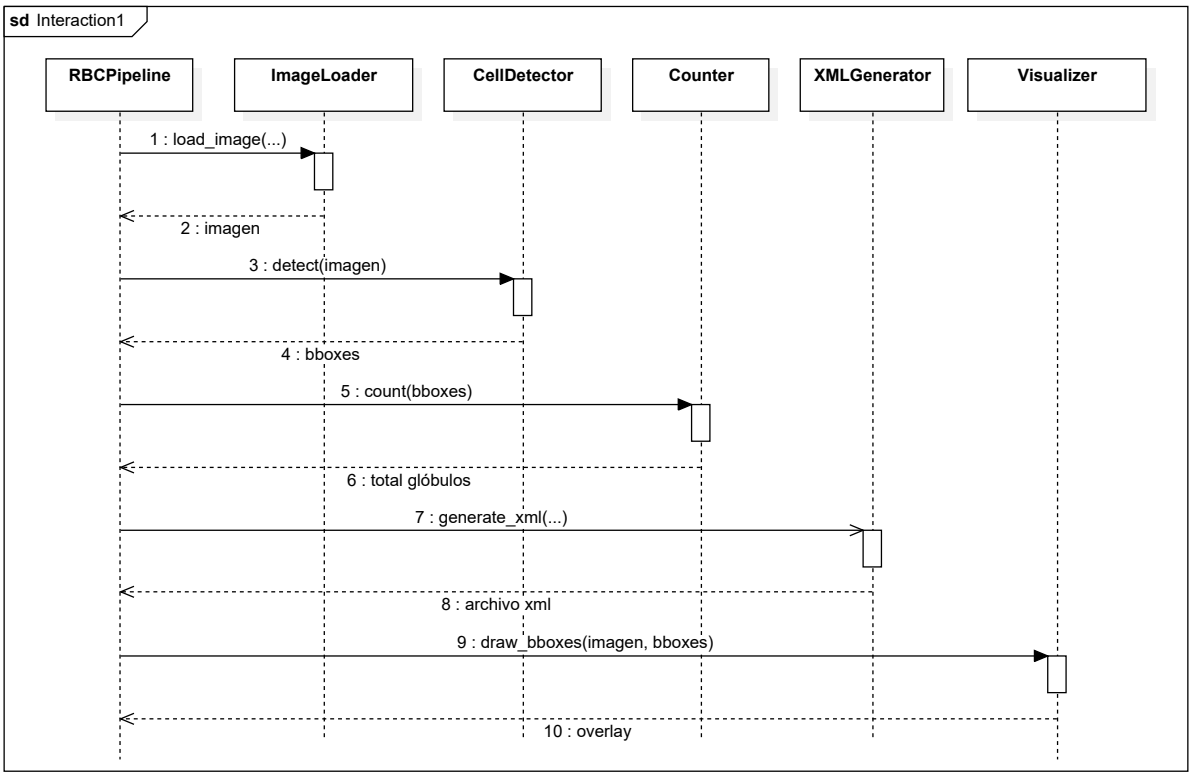


Figura 5: Diagrama UML de secuencia del algoritmo

El diagrama de secuencia de la figura 5 muestra la interacción interna entre las distintas clases del sistema durante el procesamiento de una imagen, detallando la ejecución del algoritmo de visión artificial.

El flujo de ejecución comienza en la clase `RBCPipeline`, que actúa como controlador principal del sistema. En primer lugar, solicita a `ImageLoader` la carga de la imagen de entrada mediante el método `load_image`, recibiendo como respuesta la imagen correspondiente. A continuación, la imagen se envía a la clase `CellDetector`, que aplica el modelo de detección y devuelve las bounding boxes de los glóbulos rojos detectados.

Posteriormente, la lista de detecciones es utilizada por la clase `Counter` para calcular el número total de glóbulos rojos presentes en la imagen. Seguidamente, la clase `XMLGenerator` genera un archivo XML que contiene la información estructurada de las detecciones. Finalmente, la clase `Visualizer` se encarga de dibujar las bounding boxes sobre la imagen original, generando una representación visual del resultado.

Este diagrama permite comprender cómo las distintas clases del sistema colaboran entre sí para implementar el pipeline completo de procesamiento, mostrando de forma clara el flujo de datos y la secuencia de llamadas entre los componentes.

2.3. Diagramas de actividad de los algoritmos

2.3.1. Funcionamiento general del sistema

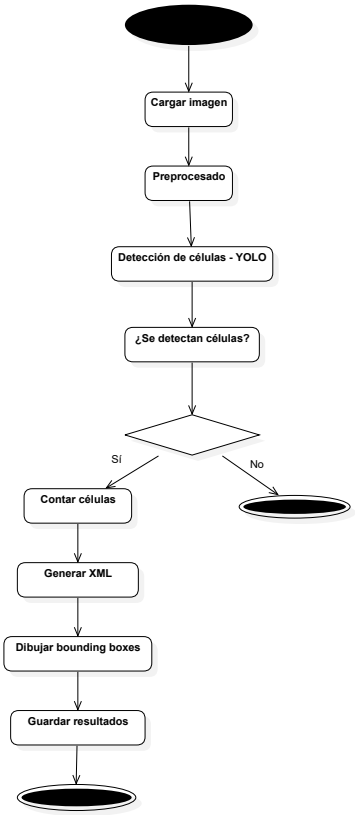


Figura 6: Diagrama UML de actividad del sistema

El diagrama de actividad de la figura 6 representa el flujo de ejecución del sistema completo para el procesamiento de imágenes de plasma sanguíneo.

El proceso comienza con la carga de la imagen, tras lo cual se realiza un preprocesado con el objetivo de preparar la imagen para su posterior análisis. A continuación, se lleva a cabo la detección de glóbulos rojos mediante el modelo de visión artificial basado en YOLO.

Posteriormente, se evalúa si se han detectado células en la imagen. En caso afirmativo, el flujo continúa con el conteo de células, seguido de la generación del archivo XML que contiene la información de las detecciones, así como de la creación de una representación visual mediante el dibujo de las bounding boxes sobre la imagen original. Finalmente, los resultados se almacenan. En caso de que no se detecten células, el proceso finaliza sin generar resultados adicionales.

2.3.2. Algoritmo de visión (YOLO)

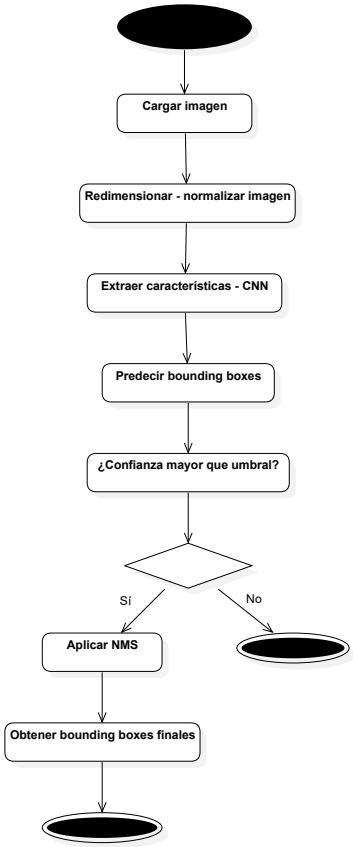


Figura 7: Diagrama UML de actividad del sistema

El diagrama de actividad de la figura 7 representa el funcionamiento del algoritmo de visión artificial utilizado para la detección de glóbulos rojos basado en el modelo YOLO.

El proceso comienza con la carga de la imagen de entrada, que es sometida a una etapa de redimensionado y normalización con el objetivo de adaptarla al formato requerido por la red neuronal. A continuación, se realiza la extracción de características mediante una red neuronal convolucional (CNN), que permite obtener una representación interna de la imagen.

Posteriormente, el modelo predice un conjunto de bounding boxes candidatas que pueden corresponder a glóbulos rojos. Estas detecciones son evaluadas en función de su confianza, filtrando aquellas que no superan un determinado umbral (0.65). En caso de que la confianza sea suficiente, se aplica el algoritmo de supresión de no máximos (Non-Maximum Suppression, NMS), con el objetivo de eliminar detecciones redundantes o solapadas.

Finalmente, se obtienen las bounding boxes definitivas que representan las células detectadas en la imagen.

2.3.3. Entrenamiento del modelo

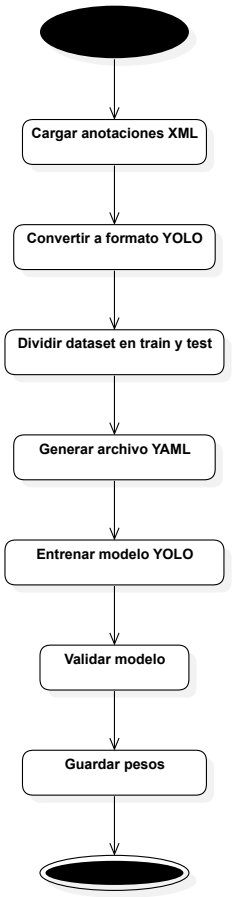


Figura 8: Diagrama UML de actividad del entrenamiento del modelo

El diagrama de actividad de la figura 8 representa el flujo de ejecución del proceso de entrenamiento del modelo de detección de glóbulos rojos.

El proceso comienza con la carga de las anotaciones en formato XML, que contienen la información de las bounding boxes correspondientes a las células. A continuación, estas anotaciones son convertidas al formato requerido por el modelo YOLO, adaptando tanto la estructura de los datos como la representación de las coordenadas.

Posteriormente, el dataset se divide en conjuntos de entrenamiento y test, lo que permite evaluar el rendimiento del modelo durante el proceso de aprendizaje. A continuación, se genera el archivo de configuración YAML, que define la estructura del dataset y las clases utilizadas, facilitando el acceso del modelo a los datos.

Una vez preparado el dataset, se procede al entrenamiento del modelo YOLO, seguido de su validación para comprobar su capacidad de generalización. Finalmente, se guardan los pesos del modelo entrenado, que serán utilizados posteriormente para la fase de inferencia.

2.3.4. Evaluación del modelo

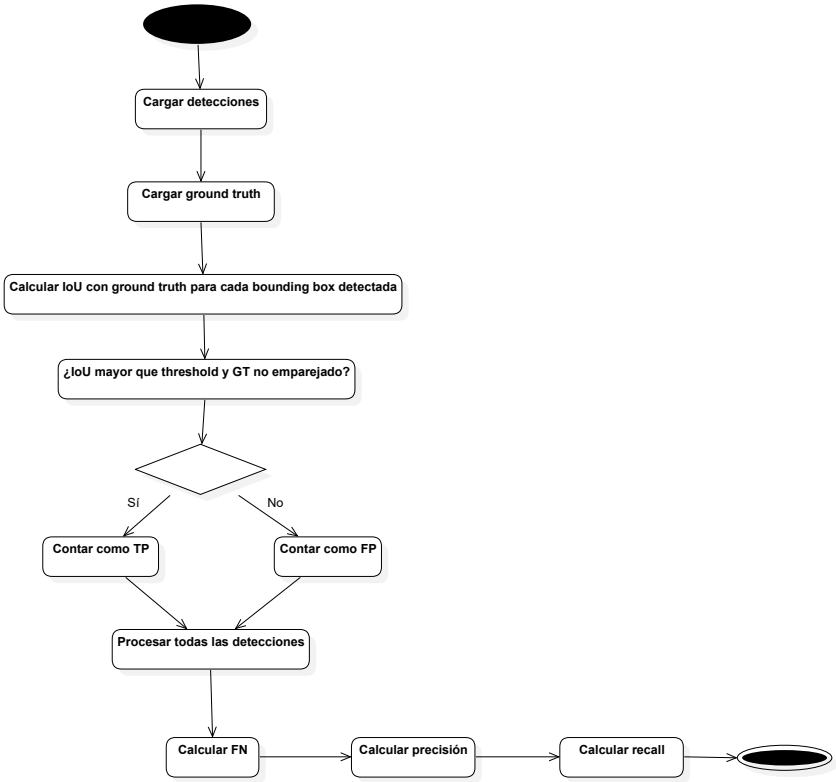


Figura 9: Diagrama UML de actividad de la evaluación con métricas

El diagrama de actividad de evaluación de la figura 9, describe las etapas del proceso de análisis de resultados una vez que ya se tienen las detecciones y el *ground truth*.

El proceso comienza con la carga de las detecciones generadas por el sistema y la carga de las anotaciones de *ground truth*. Estas dos fuentes de cajas delimitadoras son la base para la comparación: las detecciones representan lo que el detector ha encontrado y el *ground truth* representa lo que debería haber encontrado.

A continuación se calcula el grado de coincidencia entre cada bounding box detectada y las cajas de *ground truth* mediante el cálculo de la métrica *IoU (Intersection over Union)*. Este cálculo determina cuánto se solapan las cajas y permite decidir si una detección se considera correcta o no.

Después se evalúa cada detección con una condición: si el *IoU* es mayor que un umbral predefinido y la caja de *ground truth* asociada no ha sido emparejada previamente, se clasifica como verdadero positivo (TP). Si la detección no cumple esa condición, se clasifica como falso positivo (FP). Paralelamente, se identifica el conjunto de cajas de *ground truth* que no han sido emparejadas por ninguna detección para contabilizarlas como falsos negativos (FN).

Con esos conteos se calcula finalmente la precisión y el recall. La precisión mide qué proporción de las detecciones fueron realmente correctas, mientras que el recall mide qué proporción de las referencias reales fueron encontradas por el detector.